Software Requirements Specification (SRS)

Dynamic Trust Infrastructure — Hackathon MVP

Version 1.0 | September 2026

Based on PRD v1.3

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for the Dynamic Trust Infrastructure hackathon MVP. It serves as the technical reference for the development team (FastAPI backend, Next.js frontend, Neon database, Vesti escrow).

1.2 Scope

The system provides:

- A B2B Trust API for platforms
- Personal digital guarantor instruments
- Portable worker credentials
- Platform-independent jobs with escrow + micro-insurance
- Configurable trust scoring (boolean + confidence)
- Data normalisation across platform types

Out of scope: Building KYC systems, escrow on platform-integrated jobs, live insurance providers, production DID/VC, native mobile apps, machine-learning models.

1.3 Definitions

- **Trust Band**: Low / Medium / High / Building History
- **Decision**: Boolean (true/false) indicating whether a worker meets a platform's configured baseline
- **Confidence**: Numeric score reflecting strength of evidence
- **Guarantor Instrument**: Digitally signed personal guarantee by a real person related to the worker
- **Platform-Independent Job**: Job that does not run through a third-party marketplace

1.4 References

- PRD v1.3
- Insurance Act 2003 (Section 50(1) – "no premium, no cover")

- Nigeria Data Protection Act 2023 (NDPA)

2. Overall Description

2.1 Product Perspective

The system consists of:

- FastAPI backend (core logic, scoring, API)
- Next.js + React + Tailwind frontend (demo, worker views, independent job flows)
- Neon PostgreSQL database
- Vesti (escrow for platform-independent jobs only)
- External KYC provider(s) (leveraged)

2.2 User Classes

- Platform developers/integrators
- Workers (domestic / gig)
- Individuals/clients (platform-independent)
- Guarantors
- Internal demo/admin users

2.3 Operating Environment

- Backend: Python 3.11+, FastAPI
- Frontend: Modern browsers (Chrome, Firefox, Safari, Edge)
- Database: Neon (PostgreSQL)
- Hosting: Suitable for demo (e.g. Railway/Render + Vercel)

2.4 Design Constraints

- 1-month development window
- Rules-based scoring only (no ML models)
- Escrow limited to platform-independent jobs
- Personal guarantors only (real persons)
- Must support data normalisation for different platform payload structures

2.5 Assumptions

- KYC providers offer usable sandbox/test endpoints
- Vesti provides workable sandbox for escrow
- Electronic signature can be implemented as auditable “I agree” + timestamp + metadata for MVP

3. System Features & Functional Requirements

3.1 Trust Scoring Engine

- **FR-1.1** The system shall calculate a trust decision (true/false) and confidence score from work outcome signals.
- **FR-1.2** The system shall support the bands: Low, Medium, High, and “Building History”.
- **FR-1.3** Platforms shall be able to configure thresholds, weights, and required signals by platform type.
- **FR-1.4** New workers with insufficient history shall receive “Building History” status instead of a punitive score.
- **FR-1.5** The scoring formula shall be transparent and rules-based.

3.2 Data Normalisation

- **FR-2.1** The API shall accept differently structured payloads from different platform types.
- **FR-2.2** The system shall map incoming data into a common internal signal format before scoring.

3.3 Work Outcome Ingestion

- **FR-3.1** Platforms and the independent flow shall be able to post structured outcome events (completion, rating, cancellation, dispute, no-show, quality flags, etc.).
- **FR-3.2** Posted outcomes shall update the worker’s trust record and portable credentials.

3.4 Personal Guarantor Flow

- **FR-4.1** A worker shall be able to nominate a real person as guarantor.
- **FR-4.2** The system shall generate a Guarantor Instrument containing parties, scope, conditions, and validity period.
- **FR-4.3** The guarantor shall apply an electronic signature.
- **FR-4.4** Instrument status shall follow: Draft → Pending Signature → Active → Triggered / Expired / Revoked.
- **FR-4.5** Active instruments shall be queryable via API and visible in relevant UIs.
- **FR-4.6** The guarantor flow shall be available for both platform-integrated and platform-independent jobs.

3.5 Portable Credentials

- **FR-5.1** Workers shall be able to export their verified work history and trust record.
- **FR-5.2** Workers shall be able to import previously verified credentials when onboarding to a new platform.
- **FR-5.3** Imported credentials shall be usable by the Trust Scoring Engine.

3.6 Escrow & Micro-Insurance (Platform-Independent Only)

- **FR-6.1** For platform-independent jobs, the system shall place funds in escrow using Vesti.

- **FR-6.2** A micro-premium shall be payable from the escrowed funds.
- **FR-6.3** Insurance cover shall activate only after premium payment is confirmed.
- **FR-6.4** Cover shall be voided or never activated if the protected flow is abandoned.
- **FR-6.5** Escrow and micro-insurance shall **not** be available on platform-integrated jobs.

3.7 Job Completion Confirmation

- **FR-7.1** Both parties shall be able to confirm job completion.
- **FR-7.2** If the customer does not respond within a defined window, the system shall auto-release.
- **FR-7.3** If the customer actively disputes, funds shall remain held, and the case shall be flagged for review.
- **FR-7.4** Confirmed outcomes shall be written to the worker's record.

3.8 Anti-Gamification Ratings (Platform-Independent)

- **FR-8.1** Reviews may only be submitted after a completed, protected job.
- **FR-8.2** Only one review per job shall be allowed.
- **FR-8.3** Reviews shall be tied to verified identity and job ID.
- **FR-8.4** Basic rate-limiting and velocity checks shall be applied.

3.9 Identity

- **FR-9.1** The system shall integrate with existing KYC providers for NIN, BVN, liveness, and document checks.
- **FR-9.2** The system shall store references/tokens rather than raw identity data where possible.

3.10 API & Integration

- **FR-10.1** The system shall expose a clean B2B API for trust queries, outcome posting, guarantor management, and credential import/export.
- **FR-10.2** Onboarding validation shall return a decision (boolean) + confidence.
- **FR-10.3** The API shall support platform-specific configuration of scoring parameters.

3.11 Demo & UI Requirements

- **FR-11.1** A side-by-side On-Platform vs Off-Platform comparison shall be available.
- **FR-11.2** Core screens shall include: Trust card, Guarantor flow, Escrow status (independent), Completion confirmation, Worker portable record, and Individual verification result.

4. External Interface Requirements

4.1 User Interfaces

Next.js web application, responsive, desktop-first for demo purposes.

4.2 API Interface

RESTful JSON API documented with OpenAPI. Primary endpoints include:

- Trust query/onboarding validation
- Outcome ingestion
- Guarantor instrument lifecycle
- Credential export/import
- Escrow initiation & status (independent jobs)
- Job completion confirmation

4.3 Software Interfaces

- Neon PostgreSQL
- Vesti (escrow)
- External KYC provider sandbox(es)

4.4 Communication Interfaces

HTTPS only.

5. Non-Functional Requirements

5.1 Performance

- Trust query responses should feel instantaneous in the demo environment (< 1–2 seconds under normal load).

5.2 Reliability

- Core happy paths (trust query, guarantor signing, completion confirmation, escrow activation) must be stable for live demonstration.

5.3 Security

- API authentication via API keys and/or JWT.
- Electronic signatures must be auditable (timestamp, party, basic metadata).
- No real personal data of real users in demo datasets.

5.4 Usability

- Trust band and protection status must be understandable at a glance.
- Side-by-side comparison must communicate the value proposition within seconds.

5.5 Maintainability

- Scoring formula shall be isolated in a clear module.
- Normalisation logic shall be extensible for new platform types.

5.6 Compliance Considerations

- Escrow + premium flow designed to respect “no premium, no cover”.
- Data handling should follow NDPA principles (minimisation, purpose limitation). Full compliance audit is out of scope for MVP.

6. Data Requirements (High-Level)

Core entities:

- Worker
- Guarantor + Guarantor Instrument
- Work Outcome / Job
- Trust Score / Band history
- Platform (with configuration)
- Escrow Transaction (independent jobs)
- Credential Export/Import records
- Review (independent jobs)

7. Constraints & Dependencies

- Development time: approximately 1 month
- Backend must be FastAPI
- Escrow provider: Vesti
- Database: Neon
- Frontend: Next.js + React + Tailwind
- KYC and insurance services will be stubbed or sandboxed

8. Appendices

A. Trust Output Example

JSON


```
{
  "decision": true,
  "confidence": 0.82,
  "band": "High",
  "status": "Scored",
  "signals_used": ["completion_rate", "dispute_count", "repeat_hires"],
  "explanation_summary": "Strong completion history with low disputes"
}
```

B. Guarantor Instrument Status Lifecycle

Draft → Pending Signature → Active → Triggered / Expired / Revoked